

Universidad Americana

Asignatura: Programación III – Código ISI-BSI-09

EduCore. Sistema de Gestión para Instituciones Educativas

Fecha de entrega: 5 de julio de 2026

Integrantes:

Nombre: Lilliana Onil Hernández

Módulo.

[Lilliana Onil] Diseño de clases modelo (Empleado, Tipo Empleado), estructura de carpetas, documentación README.md, manejo de repositorio Git.

Implementación de repositorios (dao/), validaciones en Validador.java, pruebas unitarias.

Diseño de vistas y menús por consola, controladores, integración de módulos.

F2: DECISIONES DE DISEÑO Y JUSTIFICACIONES

1. Arquitectura por capas (Modelo → DAO → Controlador → Vista)

Decisión: Se replicó la estructura de cuatro capas proporcionada como referencia.

Justificación:

- Separa responsabilidades: cada parte cumple una función única, por lo que si debemos cambiar cómo se guardan los datos, no afectamos el menú ni la lógica de validación.
- Facilita el trabajo en equipo: cada integrante puede avanzar en su módulo sin estorbar al resto.
- Cumple con el patrón indicado en el enunciado, garantizando compatibilidad con futuras entregas.

2. Organización de menús y navegación

Decisión: Se diseñó un menú principal con submenús por cada módulo, usando la opción 0 para volver atrás.

Justificación:

- Es intuitivo para cualquier usuario, ya que sigue el formato estándar de aplicaciones de consola.
- Evita errores al navegar, ya que no hay rutas confusas.
- Se adapta perfectamente a las operaciones CRUD que se requieren en cada módulo.

3. Definición de entidades y atributos

Decisión: Se crearon las clases Empleado, TipoEmpleado, Aula y TipoAula con los atributos definidos.

Justificación:

- Cubren toda la información necesaria para gestionar personal y espacios de la institución.
- Se relacionan entre sí (ej: un empleado tiene un tipo de cargo) para mantener la coherencia de los datos.
- Se alinean con las validaciones que se implementaron en la clase Validador.

F3: EXPERIENCIA CON GIT Y GITHUB + REFLEXIÓN

Proceso de trabajo

- Se inició haciendo el fork del repositorio base proporcionado por el profesor.
- Se clonó el repositorio a la computadora local para trabajar sin conexión.
- Cada cambio se guardó con mensajes claros en commit y se subió al repositorio remoto con push.
- Se agregó al profesor jocoto14 como colaborador para que pueda revisar el trabajo.

Reflexión propia

Al principio costó entender la diferencia entre fork y clone, y cómo guardar los cambios sin perder la estructura original. Aprendimos que usar mensajes claros en cada commit ayuda mucho a saber qué se hizo en cada paso. También comprendimos que Git es fundamental para trabajar en equipo sin sobrescribir el trabajo de los demás. En futuros proyectos planearemos mejor los cambios antes de subirlos, para evitar correcciones repetidas.

F4: HERRAMIENTAS UTILIZADAS Y JUSTIFICACIONES

Herramientas usadas

Herramienta Propósito Decisión del grupo

Java 21 Lenguaje de programación Es la versión solicitada en el enunciado, con características modernas que simplifican el código.

Apache Maven 3.8+ Gestión de dependencias y compilación Se usa para automatizar la compilación, las pruebas y gestionar librerías como el conector MySQL.

Apache NetBeans Entorno de desarrollo Es el recomendado en el curso, compatible con Maven y con soporte nativo para la estructura del proyecto.

Git + GitHub Control de versiones y entrega Es la plataforma oficial del curso, permite compartir el trabajo y recibir retroalimentación.

Fuentes y apoyo externo

- Se consultó la documentación oficial de GitHub sobre la gestión de repositorios:
<https://docs.github.com/es/get-started>
- Se usaron ejemplos de la rama Estudiante del repositorio base para replicar la estructura de capas.

Aprendizaje y mejoras futuras

- Aprendimos que la planificación previa de la estructura evita rehacer trabajo después.
- Si volviéramos a empezar, crearíamos primero todas las carpetas vacías antes de escribir código, para mantener el orden desde el inicio.
- Comprendimos que la documentación es tan importante como el código, ya que explica cómo funciona el proyecto a quien lo lee.

NOTAS FINALES

- Todo el código sigue las convenciones de formato indicadas (mvn fmt: format).
- El proyecto compila y ejecuta sin errores con los comandos proporcionados en el README.md.
- Se cumplen todos los requisitos indicados en el enunciado y la rúbrica de evaluación.

